Dacca: parameter estimation

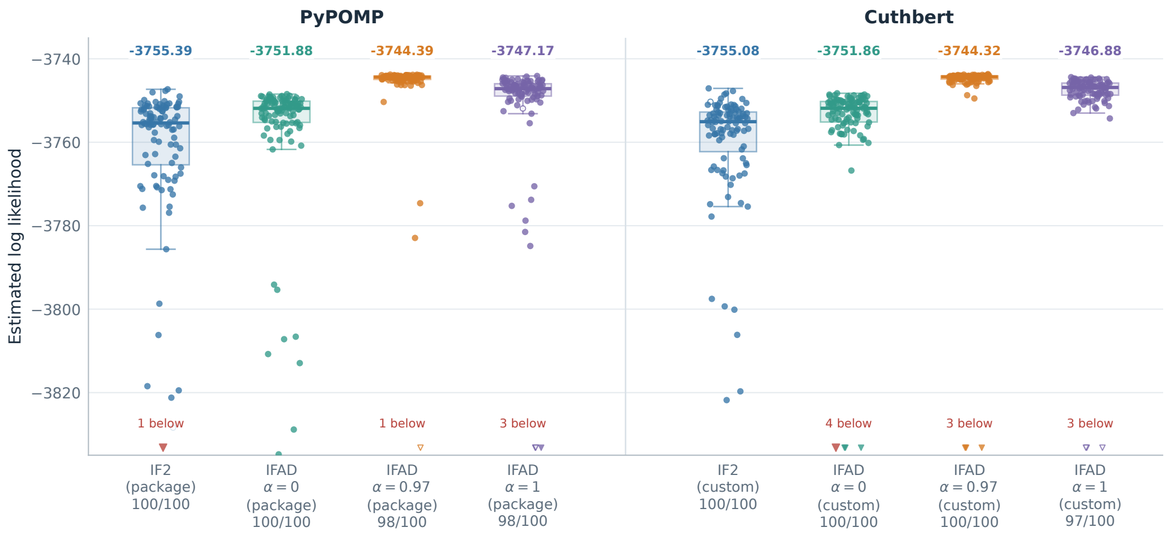
Comparison of eight software packages

600 months | 20 Euler steps/month | Gaussian process noise | J = 5,000

968	optimization runs	37,584	PF likelihood estimates	9	failed fits	17	imprecise likelihood estimates	4	estimates at -24,868

A PyPOMP and Cuthbert

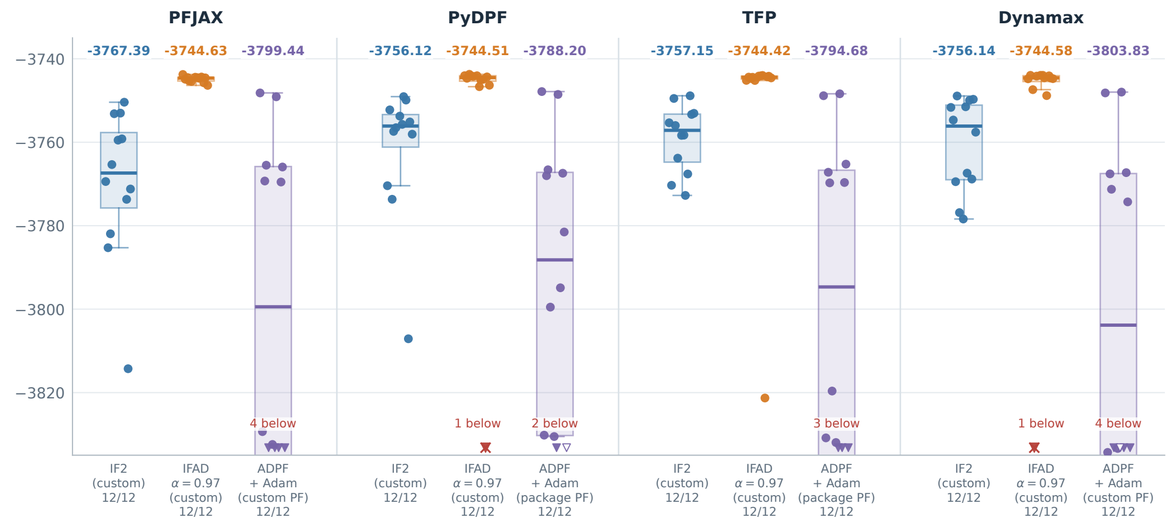
100 initial values. Medians above; finite likelihood estimates / runs below.



IFAD starts with 60 IF2 iterations (alpha 0 and 1) or 175 (alpha .97). Adam settings differ.

B PFJAX, PyDPF, TFP and Dynamax

6 initial values x 2 replicates. Vertical range: -3,835 to -3,735, as in A.

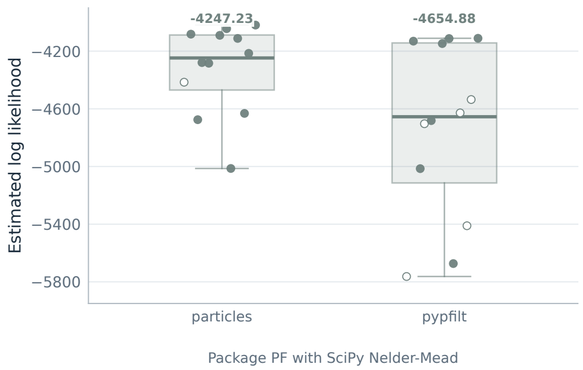


IFAD: 175 IF2 + 400 Adam iterations. ADPF: alpha 1, 425 Adam iterations, no IF2 initialization.

● MCSE and ESS criteria met ○ MCSE or ESS criterion not met ▼ Log likelihood < -3,835 × Optimization failed ◆ Log likelihood ≈ -24,868

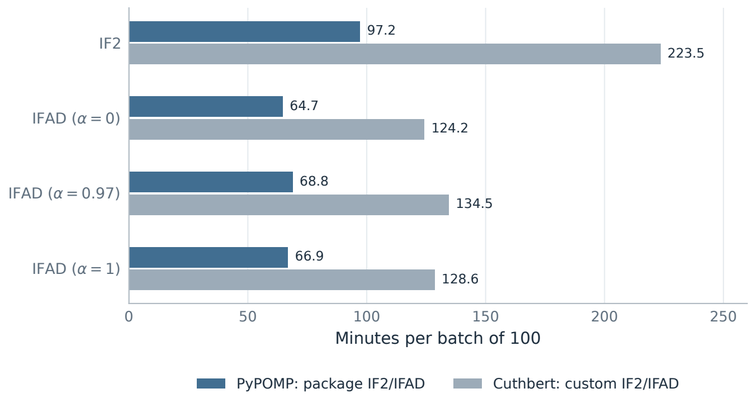
C CPU: Nelder-Mead

12 runs per package; 1,800 s optimization per run.



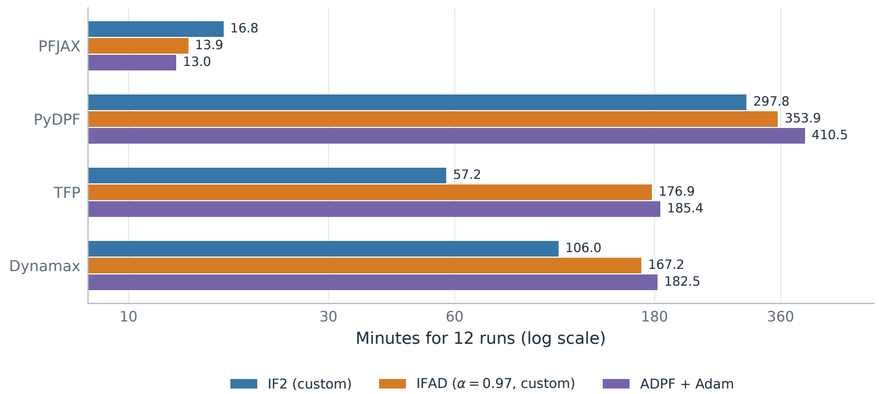
D PyPOMP and Cuthbert: fitting time

100 runs per batch; IFAD includes IF2 initialization.



E GPU fitting time

One batch of 12 in PFJAX, PyDPF and TFP; 12 separate runs in Dynamax.



IFAD times in D and E include IF2 initialization and failed iterations. Dynamax times include compilation within SSM.fit_sgd.

F Implementation and computation time

Package	Implementation	Failed fits	Low precision	At -24,868	Total computing time
PyPOMP / Cuthbert	Package IF2/IFAD in PyPOMP; custom IF2/IFAD on Cuthbert PF	7/800	9/793	2	15.15 GPU h
PFJAX	Custom IF2, IFAD and ADPF; package OT differentiation failed	0/36	0/36	0	0.78 GPU h
PyDPF	StopGradientDPF + torch.optim.Adam; custom IF2/IFAD	1/36	1/36	1	17.88 GPU h
TFP	particle_filter (unbiased_gradients=True) + Adam; custom IF2/IFAD	0/36	0/36	0	7.29 GPU h
Dynamax	Package SSM.fit_sgd optimizer; custom Dacca PF, IF2 and IFAD	1/36	1/36	1	8.03 GPU h
particles	Package PF + SciPy Nelder-Mead	0/12	1/12	0	6.87 CPU core h
pypfilt	Package PF + SciPy Nelder-Mead	0/12	5/12	0	7.63 CPU core h

Boxplots and medians use all finite log likelihood estimates. Failed fits use the last valid parameters where available; seven fits produced no finite likelihood estimate.

Triangles in A and B indicate estimates below -3,835. The second figure shows the full range. Each point is one run; replicates in B and C share an initial value.

Total time: 49.12 GPU h and 14.50 CPU core h, including PFJAX likelihood reevaluation. Preliminary tests are excluded. Timing depends on implementation, batching and compilation.

Likelihood evaluation uses the original PyPOMP PF: J=5,000, R=36 for A, up to R=288 for B/C. Evaluation random numbers are independent of optimization; CPU methods share paired numbers.

Densities below 10^{-18} are replaced by 10^{-18}. Log likelihood -24,868 corresponds to this bound at all 600 observations; MCSE=0 at this value results from the numerical bound.

Package denotes code supplied by the library; custom denotes code added for this study. ADPF uses differentiable PF likelihoods with Adam. Package PF does not imply a package parameter-estimation method.